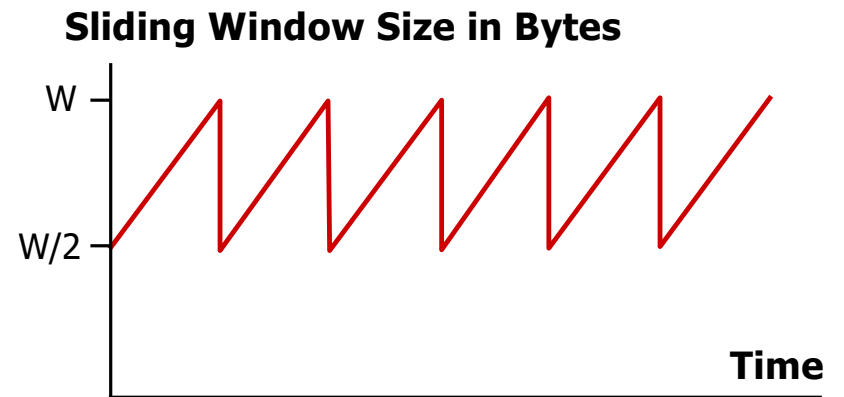


Quiz: TCP Throughput

TCP uses Additive Increase and Multiplicative Decrease (AIMD) to manage the sliding window during congestion avoidance phase, as shown in the right figure. The sliding window size indicates the # of bytes to send before waiting for ACK.

Let's ignore the slow start phase. What is the average throughput as a function of W and RTT ?

Answer: the average sliding window size is $(W/2 + W)/2 = 3W/4$. Thus, there are on average $3W/4$ bytes sent in one RTT , and the throughput is $3W/4RTT$.



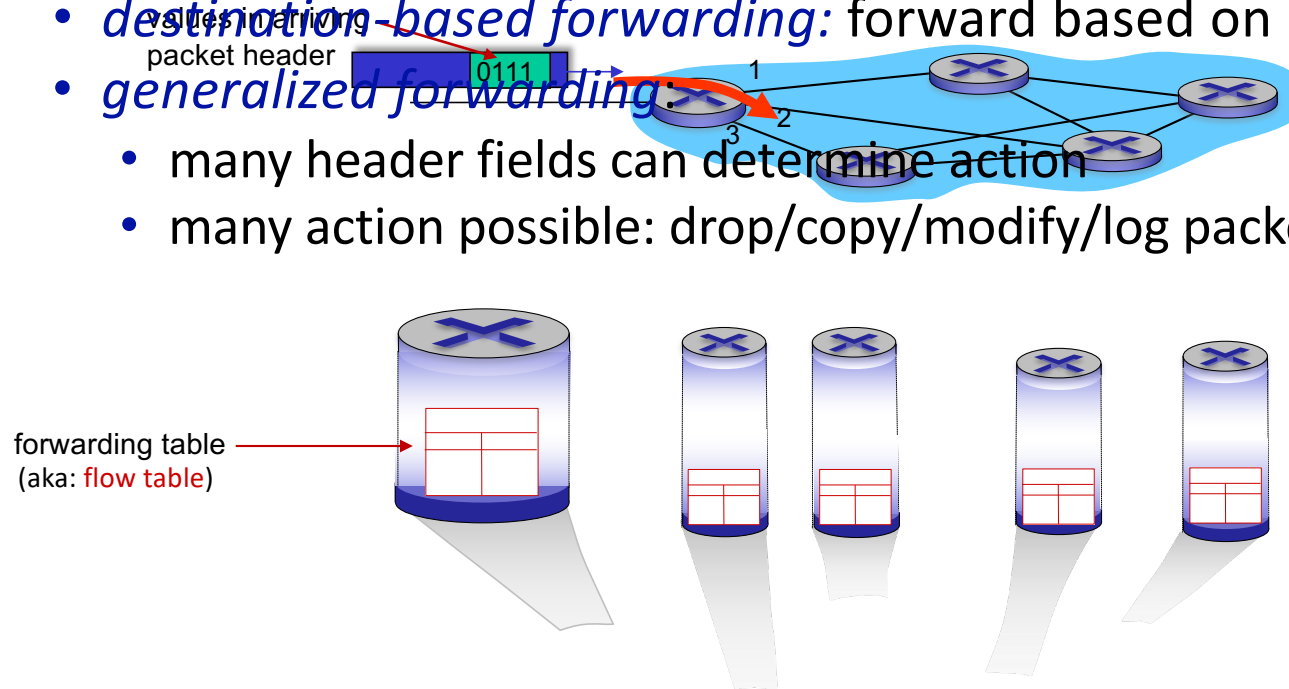
Network layer: SDN

- SDN data plane: generalized forwarding
 - Match+action, OpenFlow: match+action in action
- SDN control plane

Generalized forwarding: match plus action

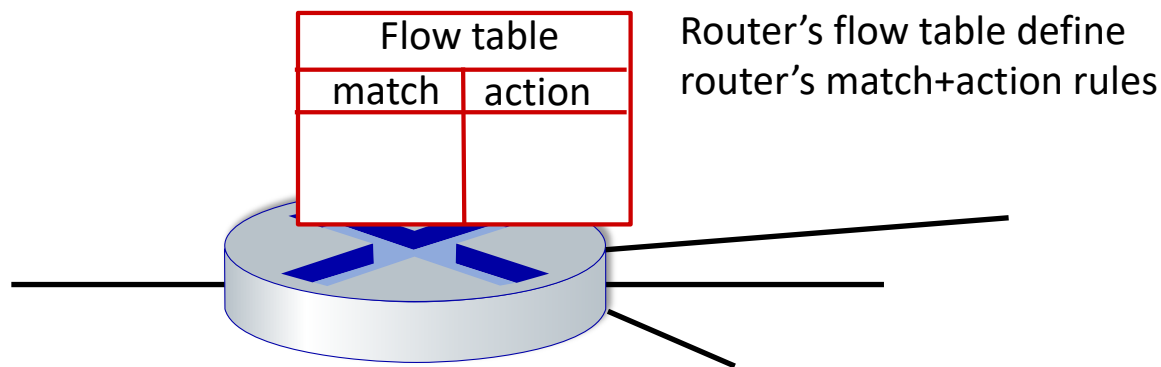
Review: each router contains a **forwarding table** (aka: **flow table**)

- “**match plus action**” abstraction: match bits in arriving packet, take action
 - **destination-based forwarding**: forward based on dest. IP address
 - **generalized forwarding**:
 - many header fields can determine action
 - many action possible: drop/copy/modify/log packet



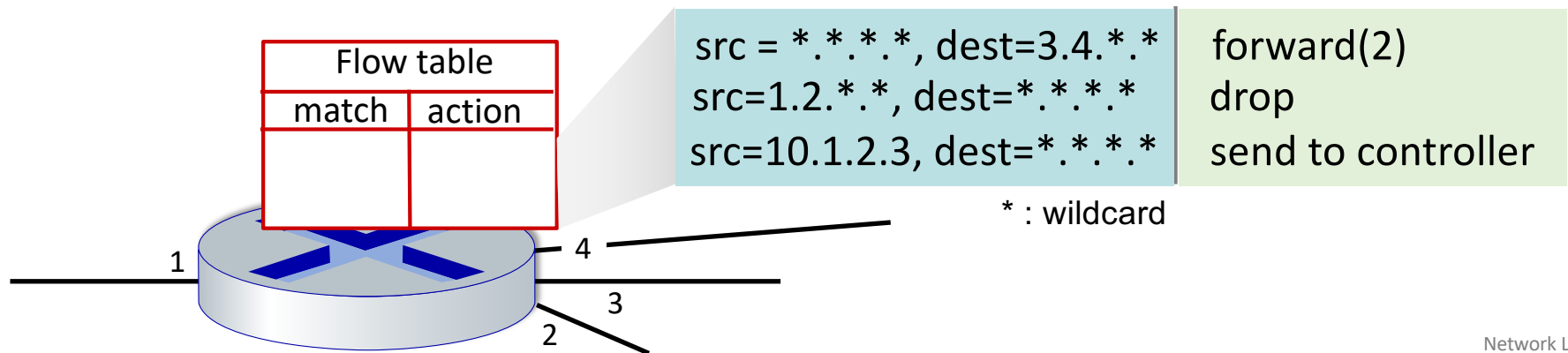
Flow table abstraction

- **flow**: defined by header field values (in link-, network-, transport-layer fields)
- **generalized forwarding**: simple packet-handling rules
 - **match**: pattern values in packet header fields
 - **actions**: for matched packet: drop, forward, modify, matched packet or send matched packet to controller
 - **priority**: disambiguate overlapping patterns
 - **counters**: #bytes and #packets

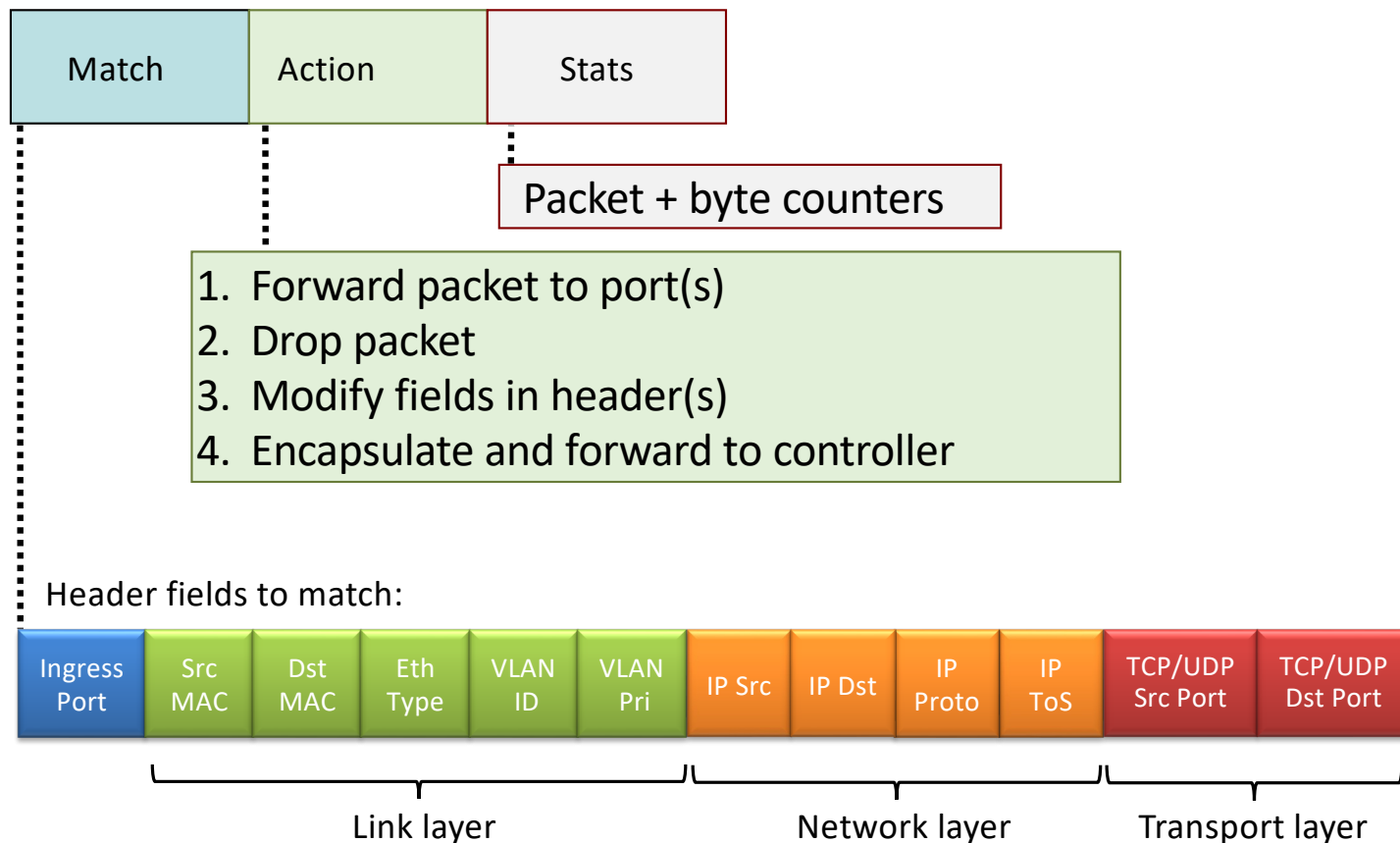


Flow table abstraction

- **flow**: defined by header fields
- **generalized forwarding: simple** packet-handling rules
 - **match**: pattern values in packet header fields
 - **actions**: for matched packet: drop, forward, modify, matched packet or send matched packet to controller
 - **priority**: disambiguate overlapping patterns
 - **counters**: #bytes and #packets



OpenFlow: flow table entries



OpenFlow: examples

Destination-based forwarding:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	*	51.6.0.8	*	*	*	*	port6

IP datagrams destined to IP address 51.6.0.8 should be forwarded to router output port 6

Firewall:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	*	*	*	*	*	22	drop

Block (do not forward) all datagrams destined to TCP port 22 (ssh port #)

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	*	*	*	*	128.119.1.1	*	*	*	*	*	drop

Block (do not forward) all datagrams sent by host 128.119.1.1

OpenFlow: examples

Layer 2 destination-based forwarding:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	VLAN Pri	IP Src	IP Dst	IP Prot	IP ToS	TCP s-port	TCP d-port	Action
*	*	22:A7:23: 11:E1:02	*	*	*	*	*	*	*	*	*	port3

layer 2 frames with destination MAC address 22:A7:23:11:E1:02 should be forwarded to output port 3

OpenFlow abstraction

- **match+action**: abstraction unifies different kinds of devices

Router

- *match*: longest destination IP prefix
- *action*: forward out a link

Switch

- *match*: destination MAC address
- *action*: forward or flood

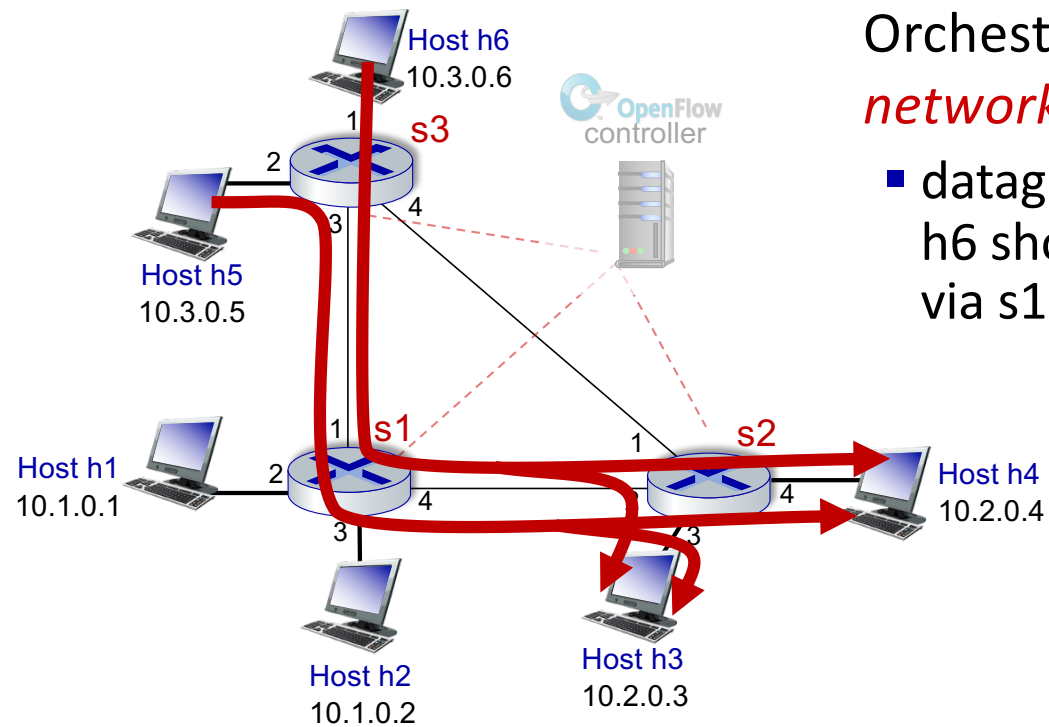
Firewall

- *match*: IP addresses and TCP/UDP port numbers
- *action*: permit or deny

NAT

- *match*: IP address and port
- *action*: rewrite address and port

OpenFlow example

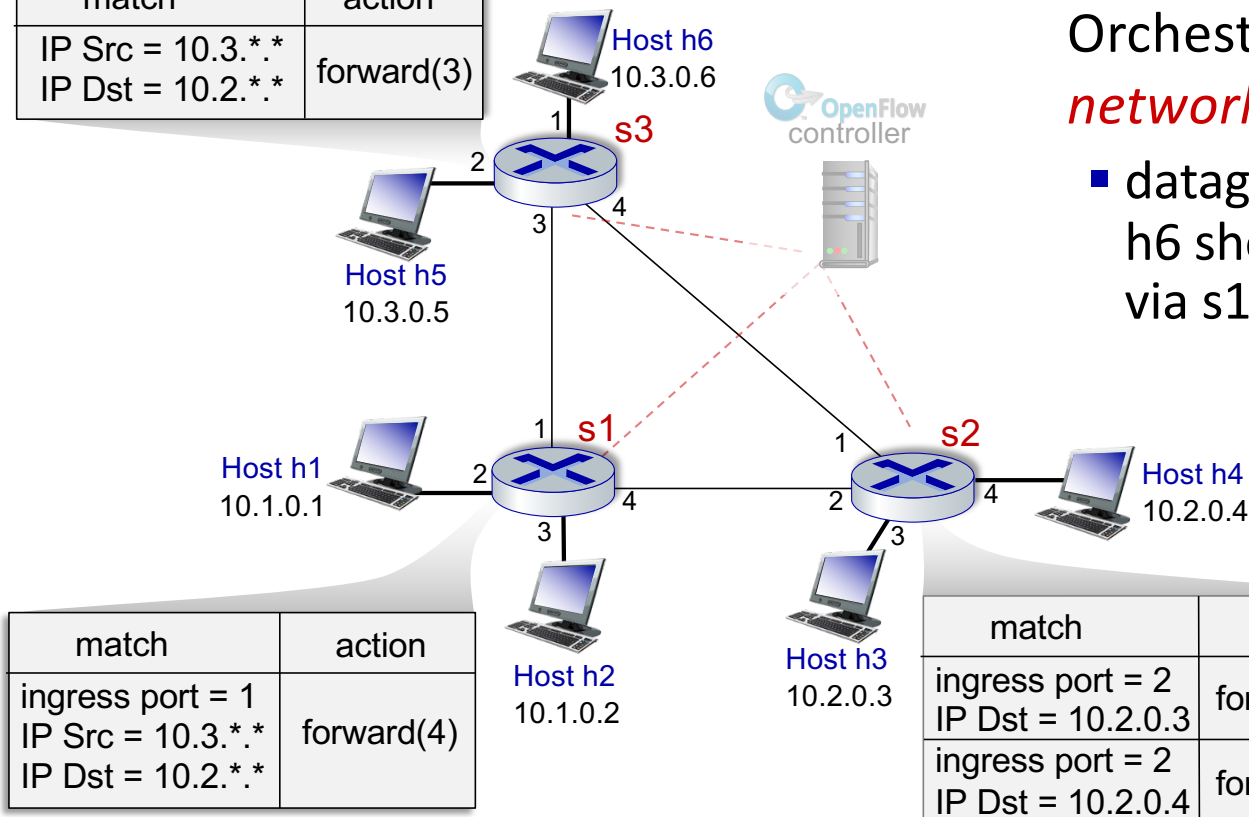


Orchestrated tables can create *network-wide* behavior, e.g.,:

- datagrams from hosts h5 and h6 should be sent to h3 or h4, via s1 and from there to s2

OpenFlow example

match	action
IP Src = 10.3.*.* IP Dst = 10.2.*.*	forward(3)



Orchestrated tables can create *network-wide* behavior, e.g.,:

- datagrams from hosts h5 and h6 should be sent to h3 or h4, via s1 and from there to s2

Generalized forwarding: summary

- “match plus action” abstraction: match bits in arriving packet header(s) in any layers, take action
 - matching over many fields (link-, network-, transport-layer)
 - local actions: drop, forward, modify, or send matched packet to controller
 - “program” *network-wide* behaviors
- simple form of “network programmability”
 - programmable, per-packet “processing”
 - *historical roots*: active networking
 - *today*: more generalized programming: P4 (see p4.org).

Network layer: SDN

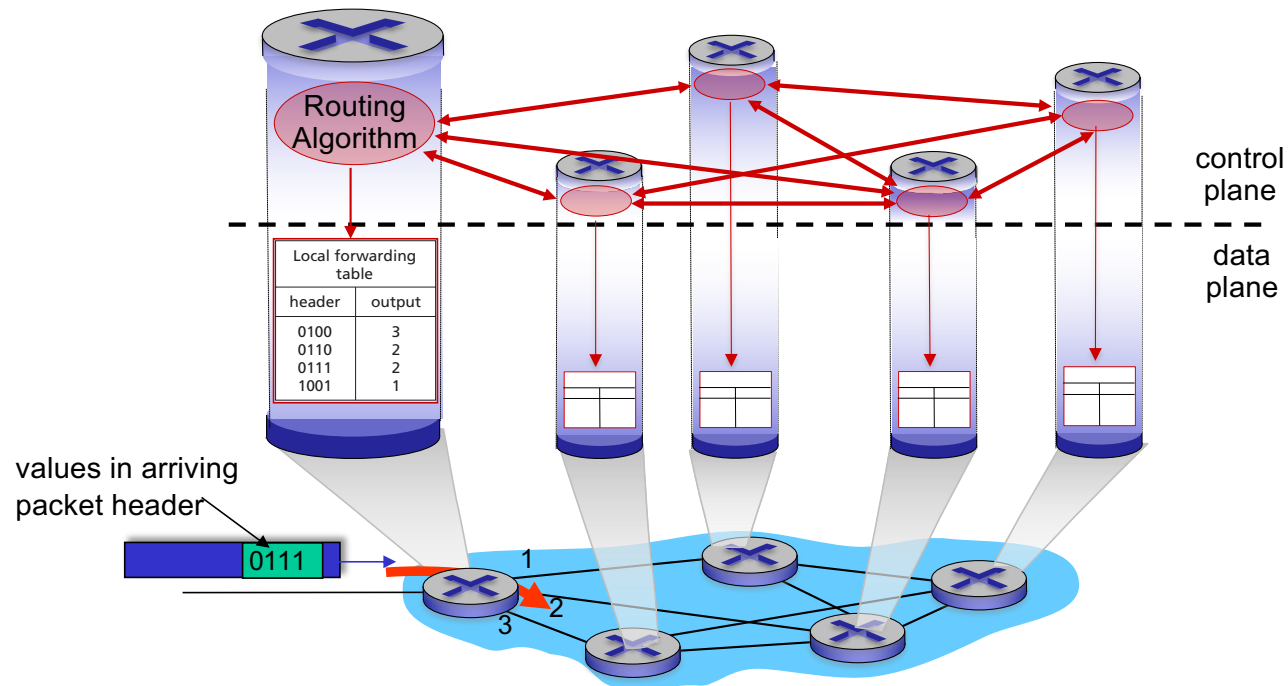
- SDN data plane: generalized forwarding
 - Match+action, OpenFlow: match+action in action
- SDN control plane

Software defined networking (SDN)

- Internet network layer: historically implemented via distributed, per-router control approach:
 - *monolithic* router contains switching hardware, runs proprietary implementation of Internet standard protocols (IP, RIP, IS-IS, OSPF, BGP) in proprietary router OS (e.g., Cisco IOS)
 - different “middleboxes” for different network layer functions: firewalls, load balancers, NAT boxes, ..
- ~2005: renewed interest in rethinking network control plane

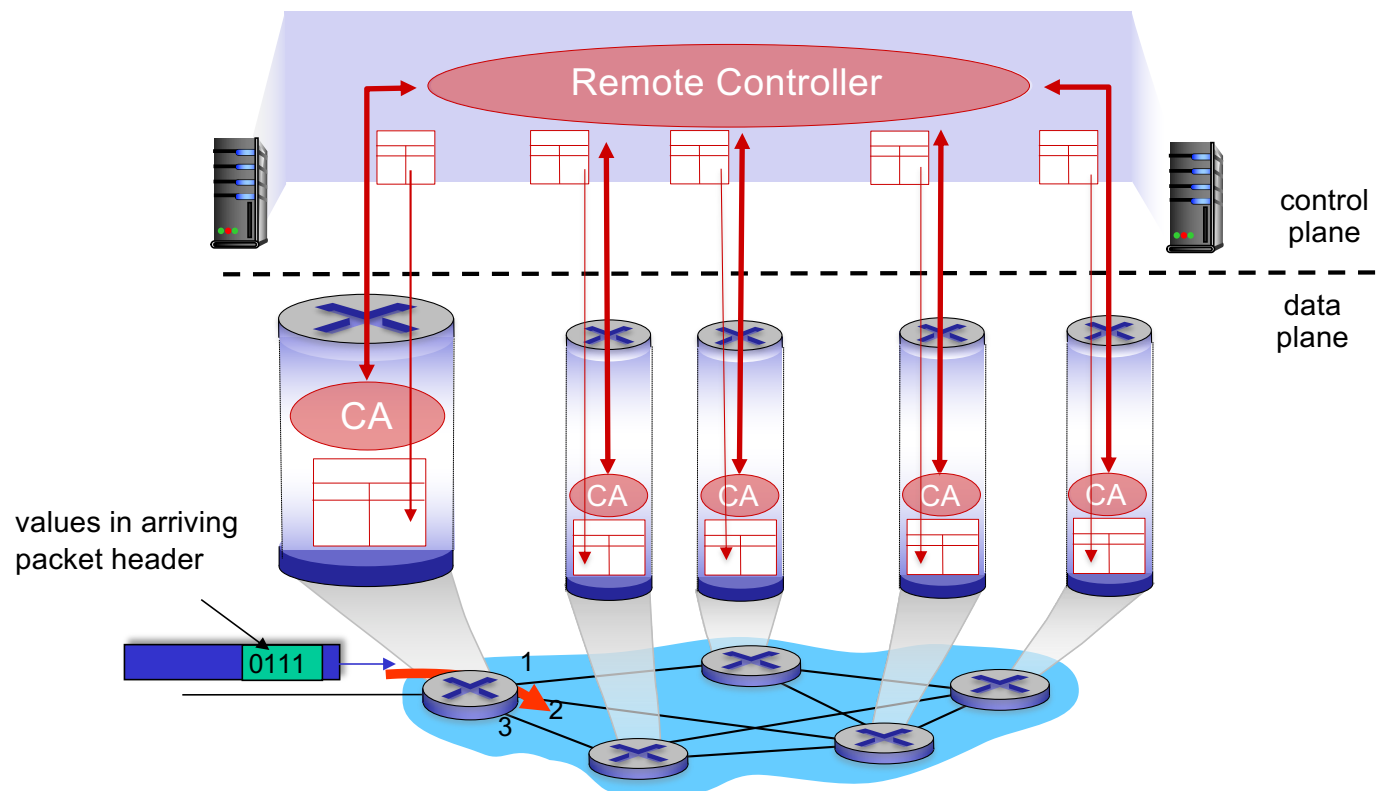
Per-router control plane

Individual routing algorithm components *in each and every router* interact in the control plane to compute forwarding tables



Software-Defined Networking (SDN) control plane

Remote controller computes, installs forwarding tables in routers

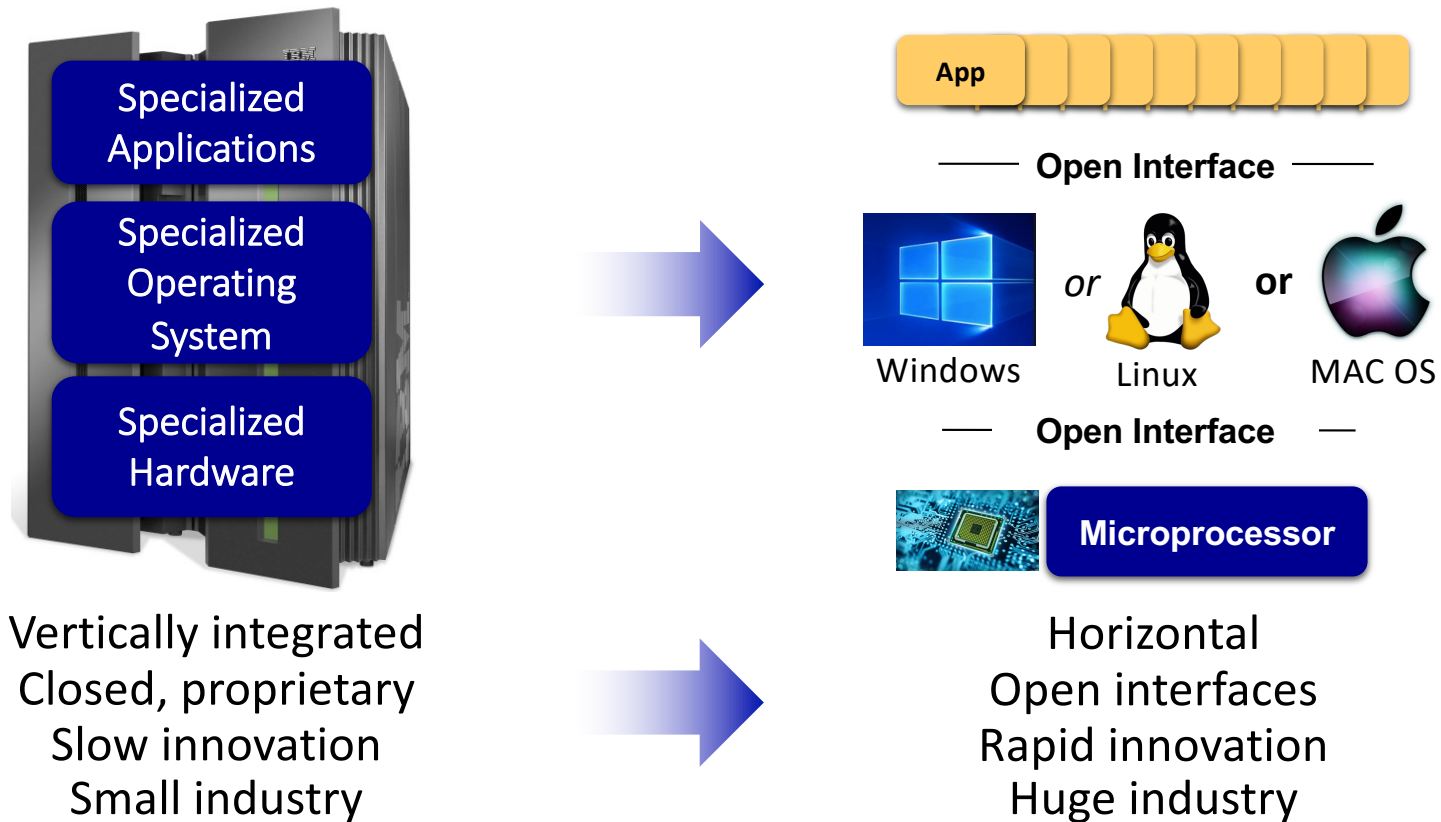


Software defined networking (SDN)

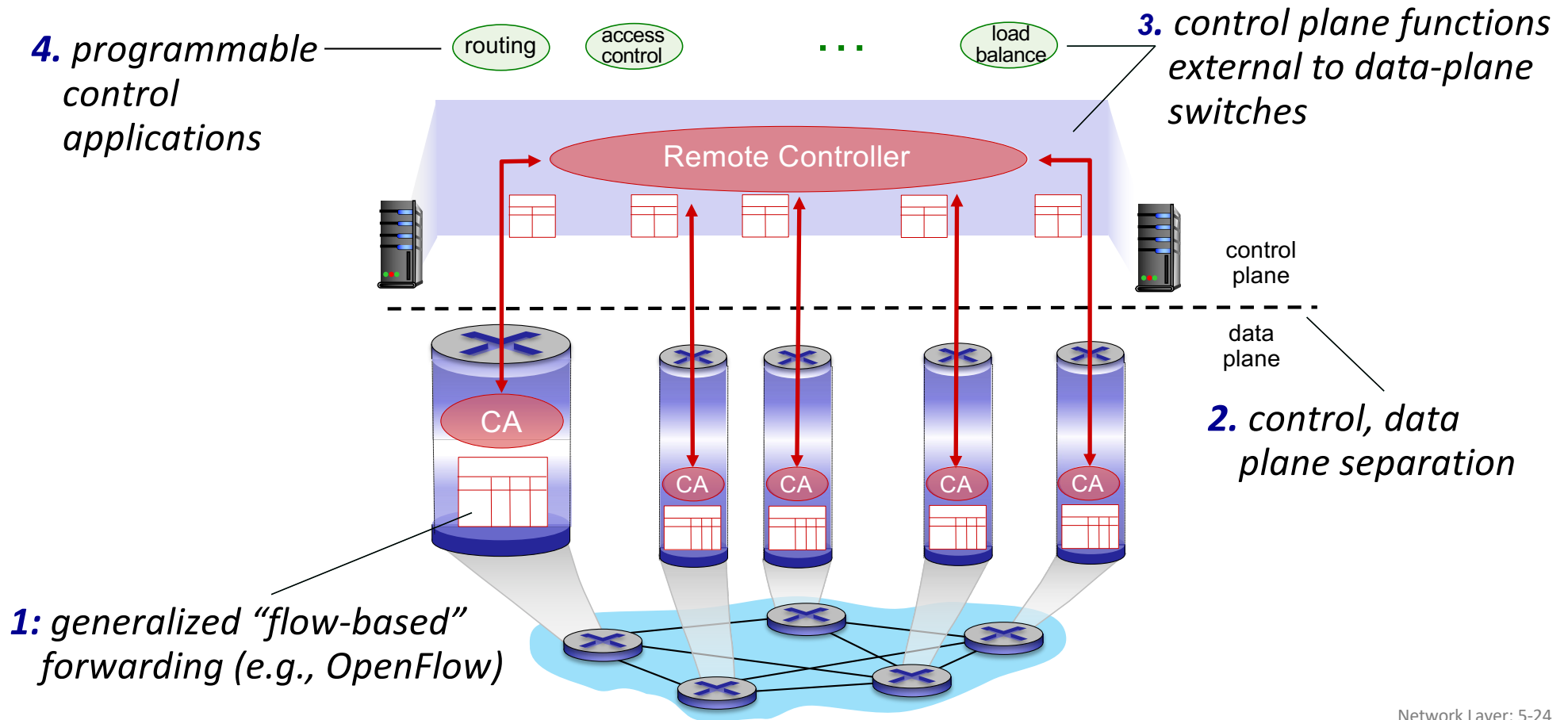
Why a *logically centralized* control plane?

- easier network management: avoid router misconfigurations, greater flexibility of traffic flows
- table-based forwarding (recall OpenFlow API) allows “programming” routers
 - centralized “programming” easier: compute tables centrally and distribute
 - distributed “programming” more difficult: compute tables as result of distributed algorithm (protocol) implemented in each-and-every router
- open (non-proprietary) implementation of control plane
 - foster innovation: let 1000 flowers bloom

SDN analogy: mainframe to PC revolution



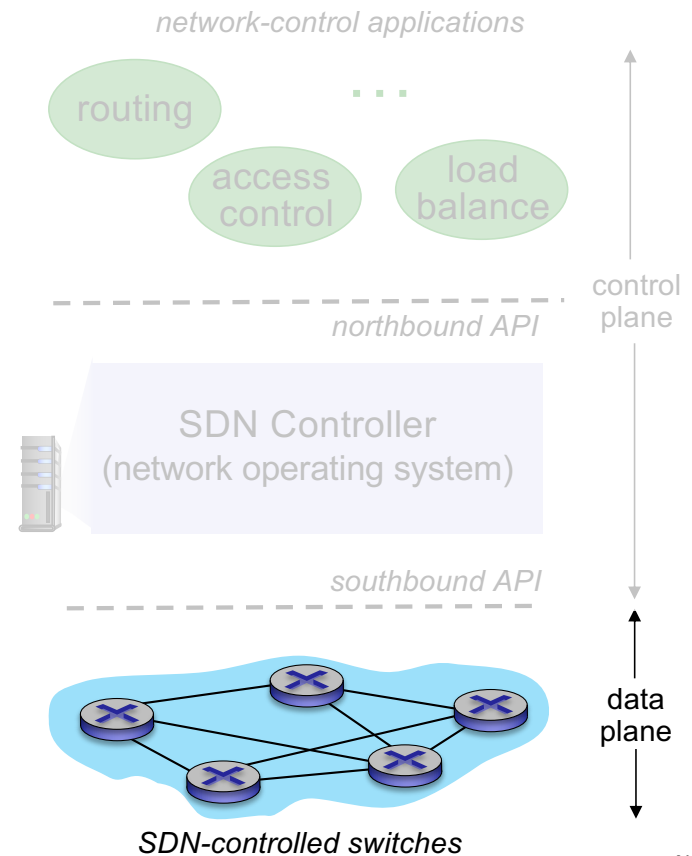
Software defined networking (SDN)



Software defined networking (SDN)

Data-plane switches:

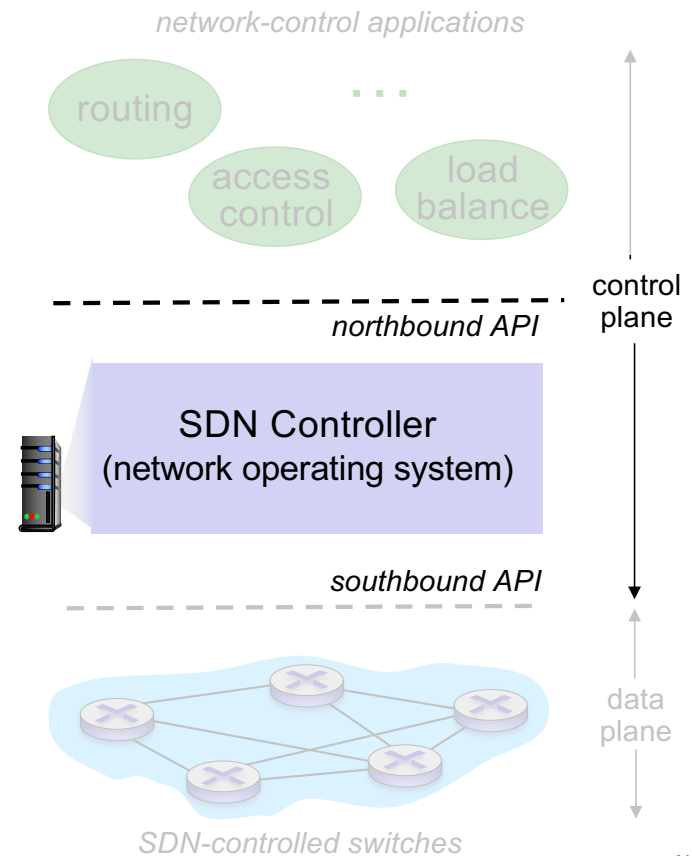
- fast, simple, commodity switches implementing generalized data-plane forwarding in hardware
- flow (forwarding) table computed, installed under controller supervision
- API for table-based switch control (e.g., OpenFlow)
 - defines what is controllable, what is not
- protocol for communicating with controller (e.g., OpenFlow)



Software defined networking (SDN)

SDN controller (network OS):

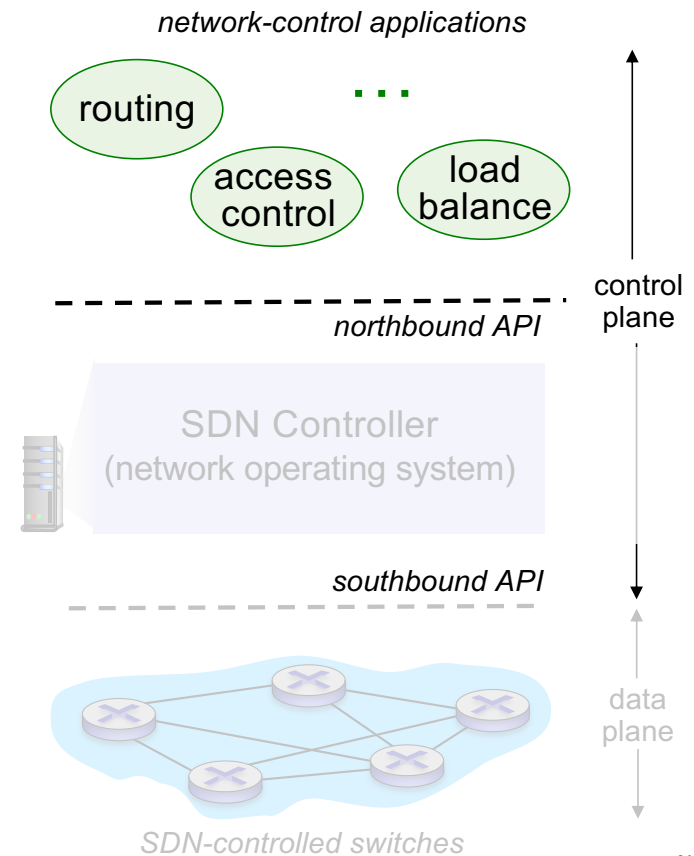
- maintain network state information
- interacts with network control applications “above” via northbound API
- interacts with network switches “below” via southbound API
- implemented as distributed system for performance, scalability, fault-tolerance, robustness



Software defined networking (SDN)

network-control apps:

- “brains” of control: implement control functions using lower-level services, API provided by SDN controller
- *unbundled*: can be provided by 3rd party: distinct from routing vendor, or SDN controller

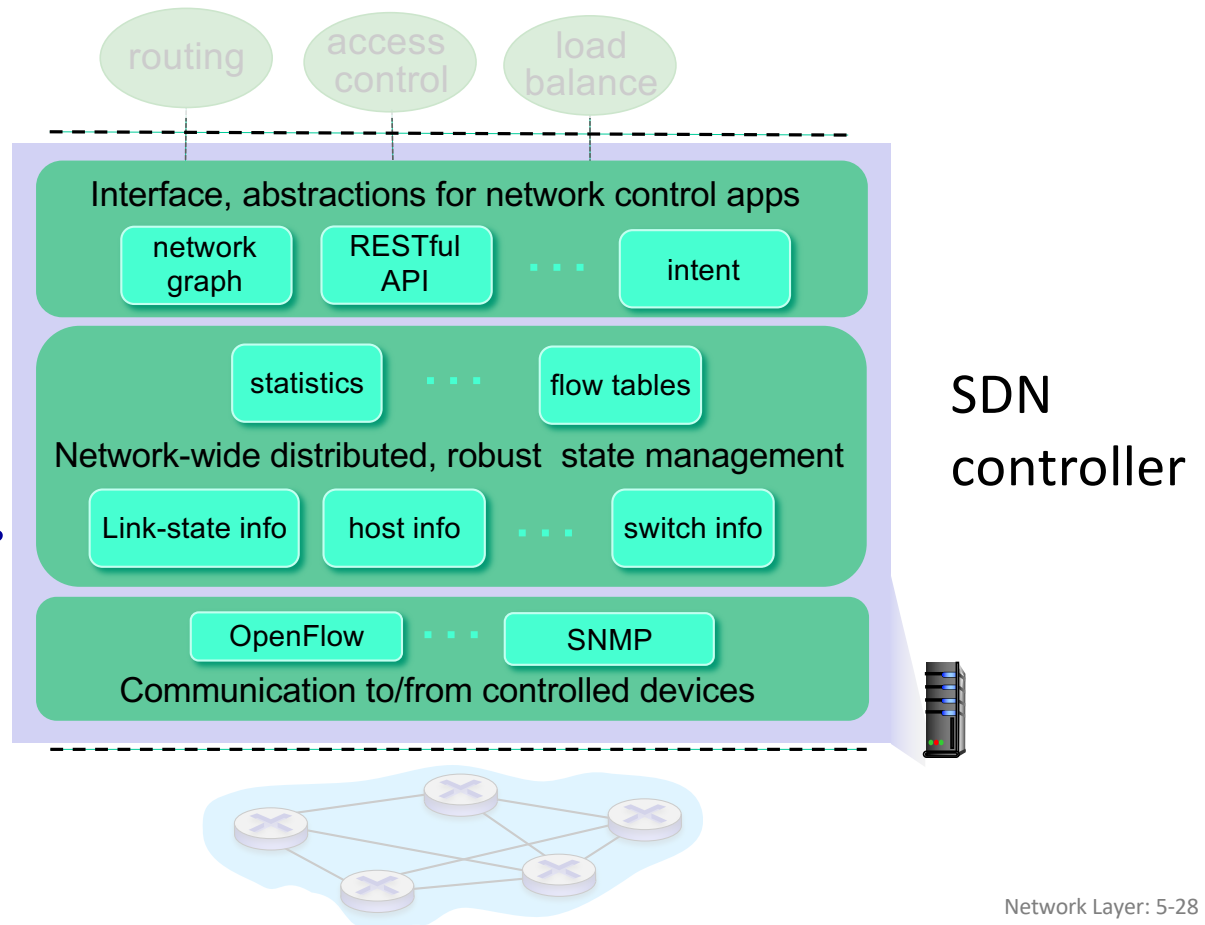


Components of SDN controller

interface layer to network control apps: abstractions API

network-wide state management: state of networks links, switches, services: a *distributed database*

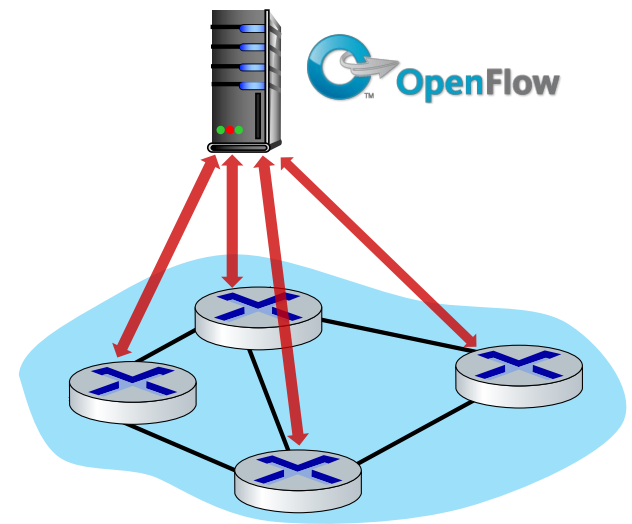
communication: communicate between SDN controller and controlled switches



OpenFlow protocol

- operates between controller, switch
- TCP used to exchange messages
 - optional encryption
- three classes of OpenFlow messages:
 - controller-to-switch
 - asynchronous (switch to controller)
 - symmetric (misc.)
- distinct from OpenFlow API
 - API used to specify generalized forwarding actions

OpenFlow Controller

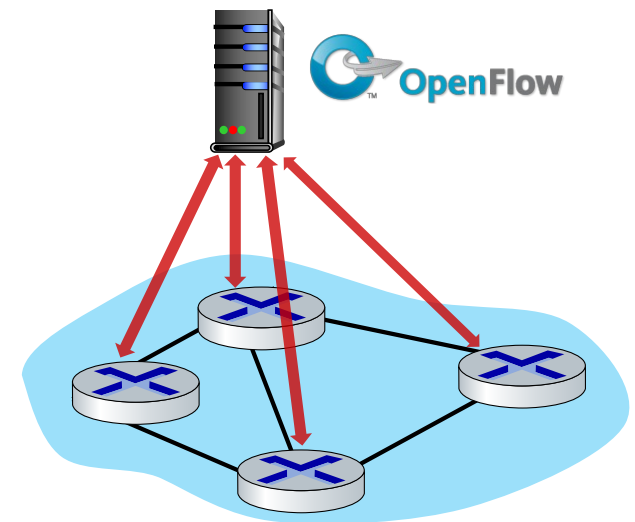


OpenFlow: controller-to-switch messages

Key controller-to-switch messages

- *features*: controller queries switch features, switch replies
- *configure*: controller queries/sets switch configuration parameters
- *modify-state*: add, delete, modify flow entries in the OpenFlow tables
- *packet-out*: controller can send this packet out of specific switch port

OpenFlow Controller

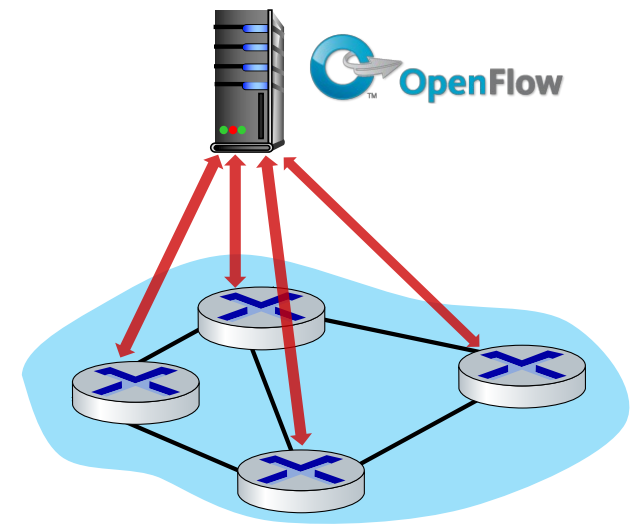


OpenFlow: switch-to-controller messages

Key switch-to-controller messages

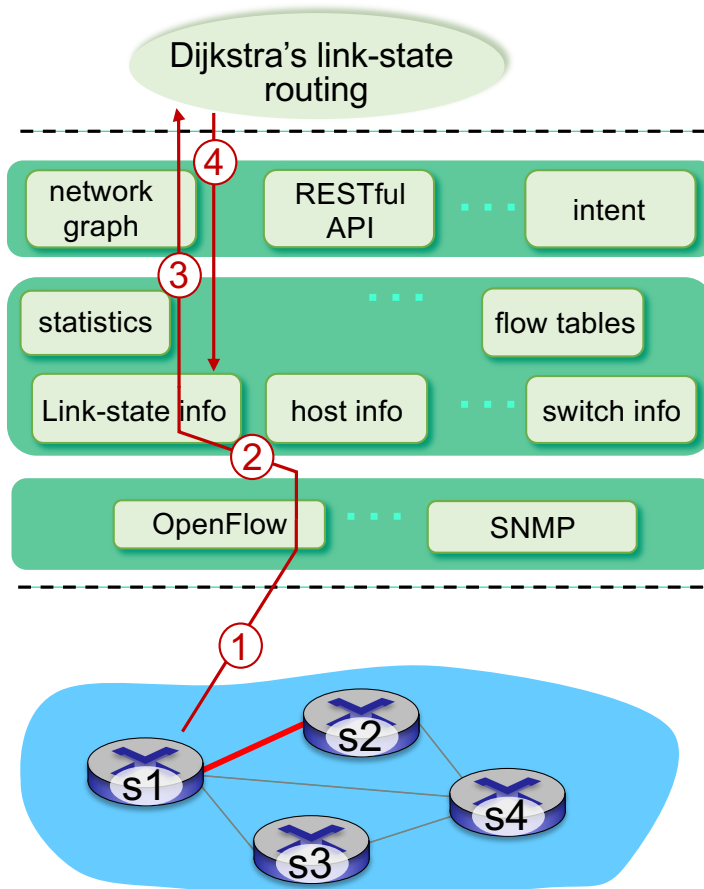
- *packet-in*: transfer packet (and its control) to controller. See packet-out message from controller
- *flow-removed*: flow table entry deleted at switch
- *port status*: inform controller of a change on a port.

OpenFlow Controller



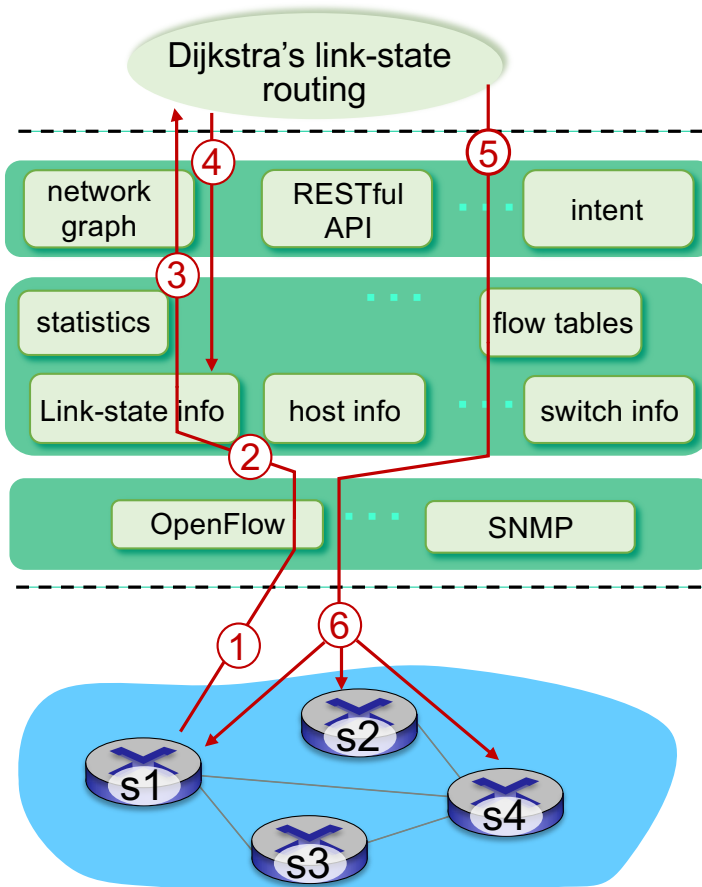
Fortunately, network operators don't "program" switches by creating/sending OpenFlow messages directly. Instead use higher-level abstraction at controller

SDN: control/data plane interaction example



- ① S1, experiencing link failure uses OpenFlow port status message to notify controller
- ② SDN controller receives OpenFlow message, updates link status info
- ③ Dijkstra's routing algorithm application has previously registered to be called when ever link status changes. It is called.
- ④ Dijkstra's routing algorithm access network graph info, link state info in controller, computes new routes

SDN: control/data plane interaction example

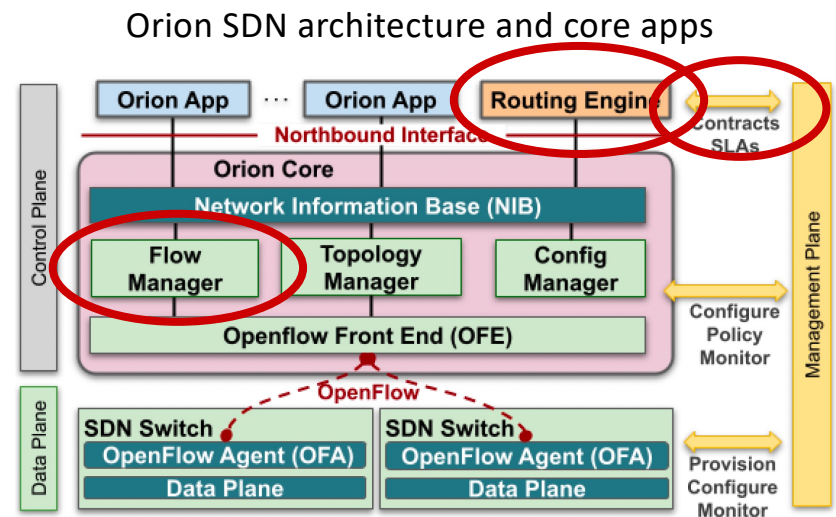


- ⑤ link state routing app interacts with flow-table-computation component in SDN controller, which computes new flow tables needed
- ⑥ controller uses OpenFlow to install new tables in switches that need updating

Google ORION SDN control plane

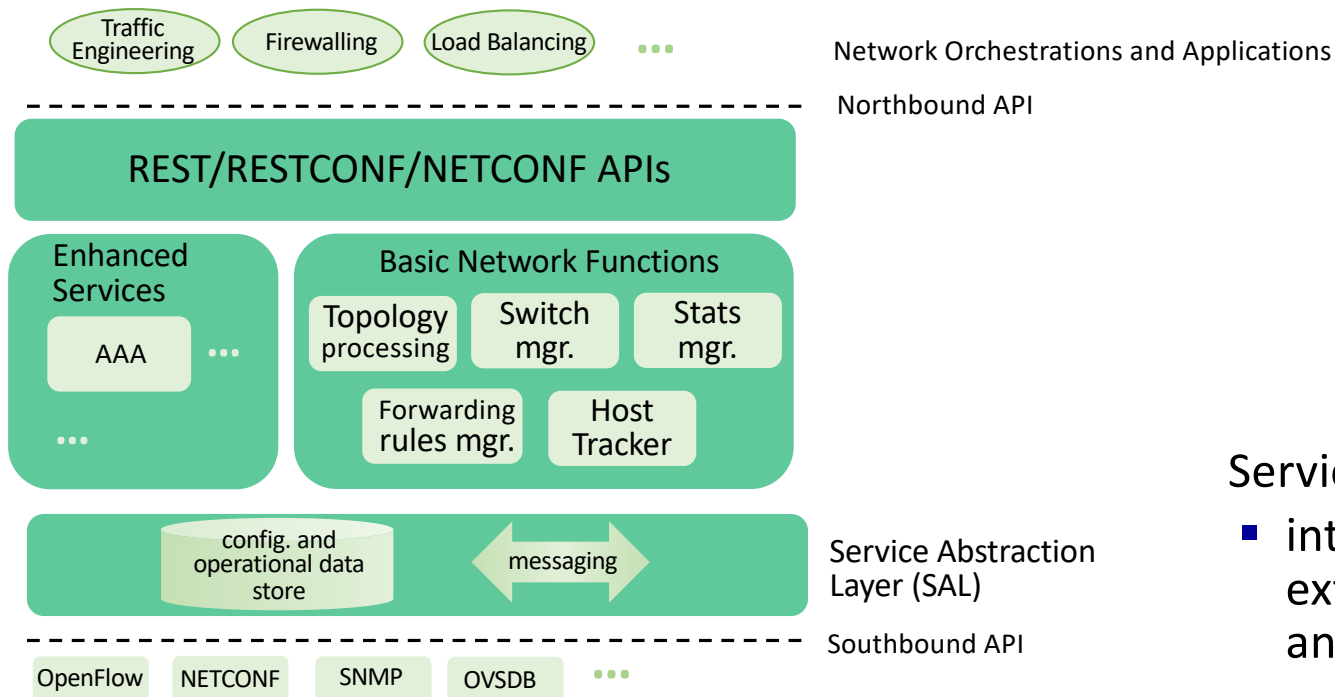
ORION: Google's SDN control plane (*NSDI'21*): control plane for Google's datacenter (Jupiter) and wide area (B4) networks

- **routing** (intradomain, iBGP), traffic engineering: implemented in *applications* on top of ORION core
- **edge-edge flow-based** controls (e.g., CoFlow scheduling) to meet contract SLAs
- **management**: pub-sub distributed microservices in Orion core, OpenFlow for switch signaling/monitoring



Note: ORION provides *intradomain* services within Google's network

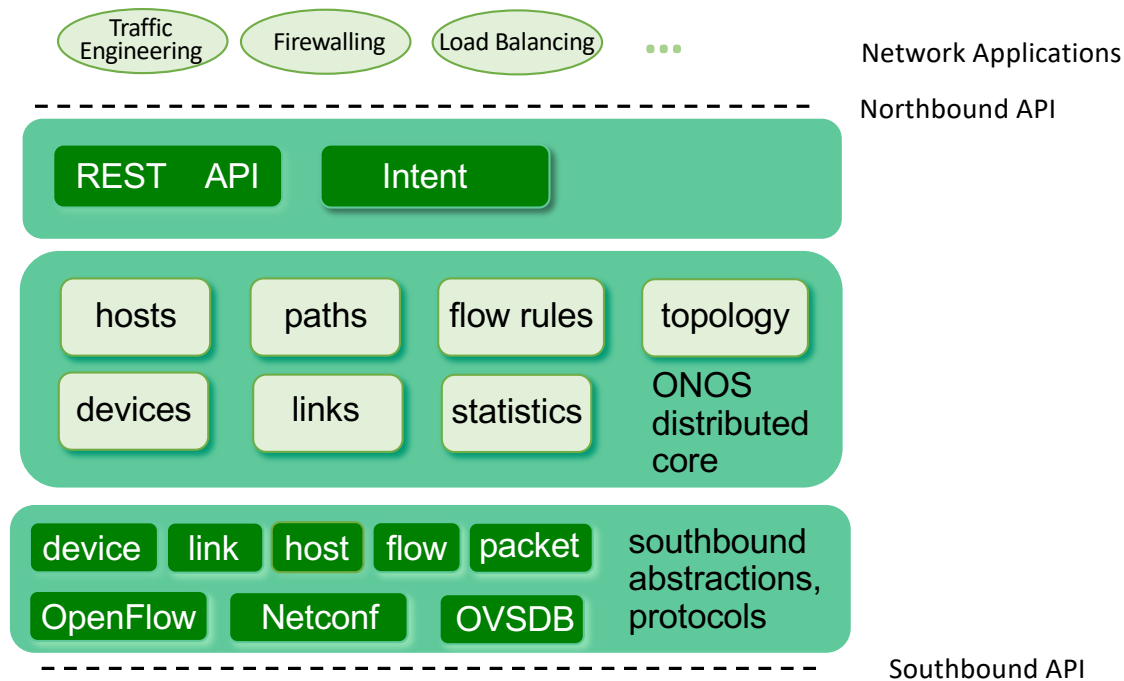
OpenDaylight (ODL) controller



Service Abstraction Layer:

- interconnects internal, external applications and services

Open Network OS (ONOS) controller



- control apps separate from controller
- intent framework: high-level specification of service: what rather than how
- considerable emphasis on distributed core: service reliability, replication performance scaling

SDN: selected challenges

- hardening the control plane: dependable, reliable, performance-scalable, secure distributed system
 - robustness to failures: leverage strong theory of reliable distributed system for control plane
 - dependability, security: “baked in” from day one?
- networks, protocols meeting mission-specific requirements
 - e.g., real-time, ultra-reliable, ultra-secure
- Internet-scaling: beyond a single AS
- SDN critical in 5G cellular networks

CSEE4199: COMPUTER NETWORKS

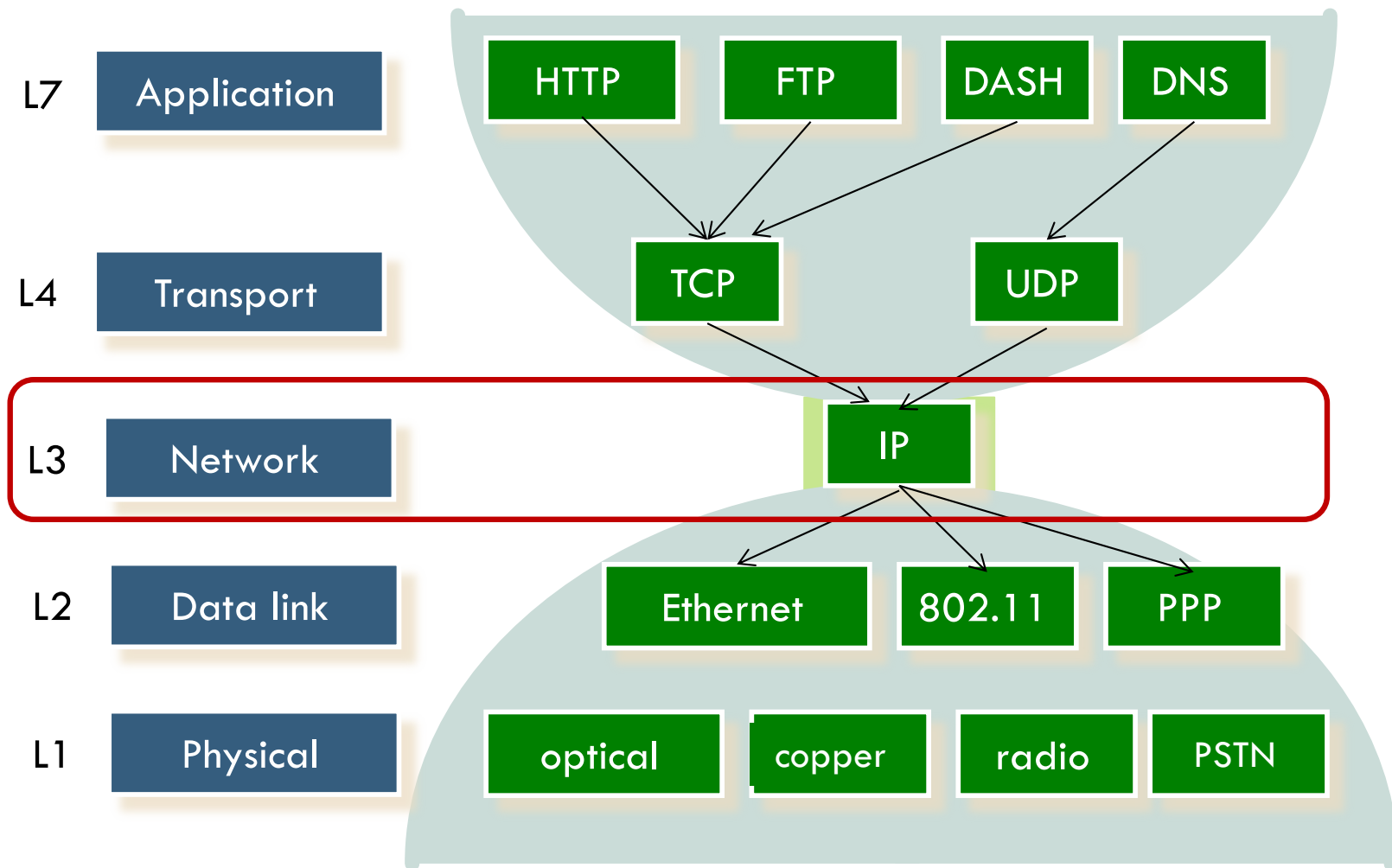
DATA LINK LAYER

Apr. 2, 2026

Xia Zhou

Link Layer

2



From a Network to a Single Link

3

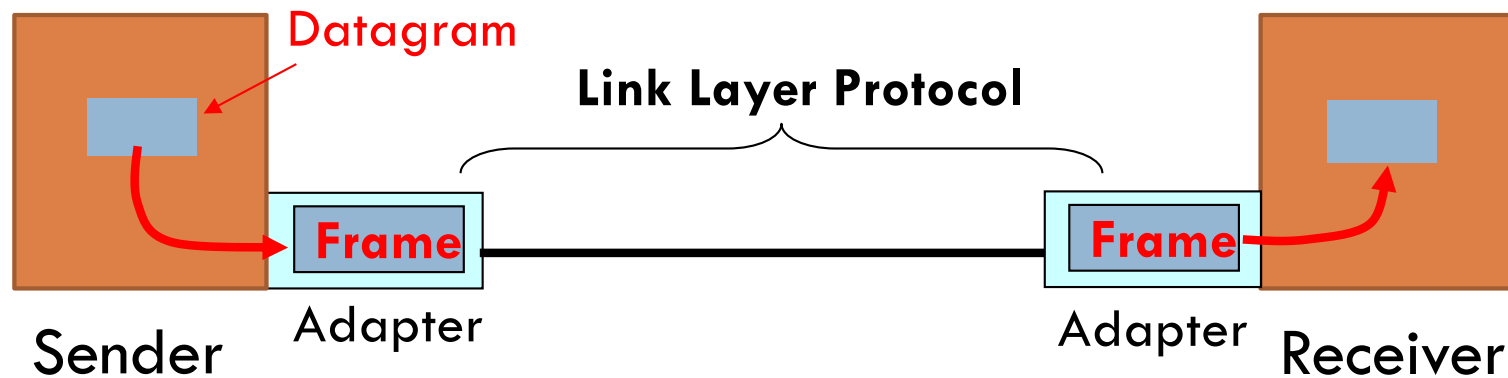
- Each hop may use different link technology
 - ▣ Ethernet, 802.11 wireless LAN, ...

- Link layer protocol for each hop
 - ▣ Hide the detail of link technology from upper layers

Data Link Layer

4

- Implemented in network interface card (adapter)
 - ▣ Ethernet card, 802.11 card, PC card



- Encapsulate datagram in a frame
- Add error checking bits, flow control

- Look for errors, flow control
- Extract datagram and pass it to upper layer

Data Link Layer Services

5

□ Framing

- Add header and trailer, encapsulate packet into frame

□ Error detection and correction

- Errors caused by signal attenuation, noise

□ Medium access control

- Who can access the medium and when

□ Reliable delivery

- Local retransmission upon frame losses
- Seldom used on low-bit error links

Error Detection

10

- Errors are unavoidable
 - ▣ Electrical interference, thermal noise...
- **Solution:** transmit **extra (redundant)** information
 - ▣ A tradeoff between accuracy and overhead
- Detection techniques
 - ▣ Parity checking
 - ▣ Checksum
 - ▣ Cyclic redundancy check (CRC)

Error Detection: Parity Check

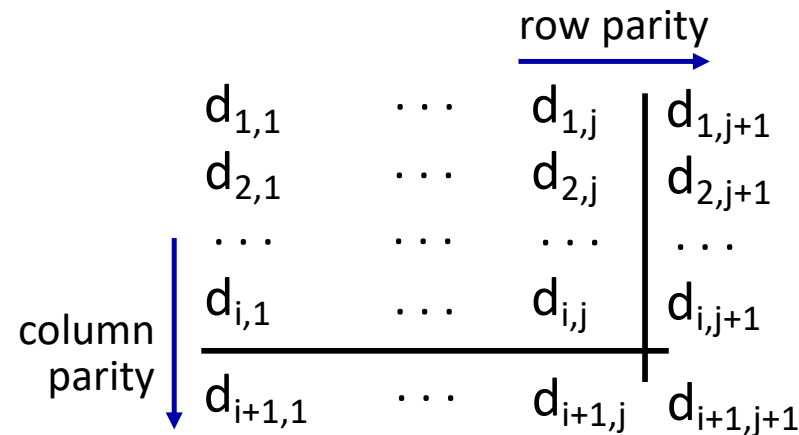
11

- Add an extra bit to a 7-bit code
- **Odd parity:** ensure an odd number of 1s
 - e.g., 0101011 → 0101011**1**
- **Even parity:** ensure an even number of 1s
 - e.g., 0101011 → 0101011**0**
- Fail to deal with multiple bit errors

Error Detection: Parity Check

12

- two-dimensional parity: detect *and correct* single bit errors



Assume even parity

no errors: $\begin{array}{ccccc|c} 1 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 \\ \hline 1 & 0 & 1 & 0 & 1 & 0 \end{array}$

detected
and
correctable
single-bit
error:

$\begin{array}{ccccc|c} 1 & 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 1 \\ \hline 1 & 0 & 1 & 0 & 1 & 0 \end{array}$ parity error

Error Detection: Checksum

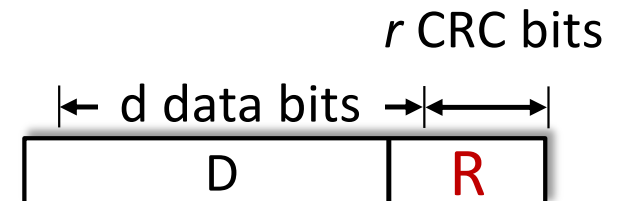
13

- Treat data as a sequence of 16-bit words
- Compute a sum of all the words, with no carries
- Transmit the sum along with the packet
- Similar as TCP checksum

Error Detection: Cyclic Redundancy Check (CRC)

14

- **Key idea:** divide the information by a number, and take the remainder as the check value
 - ▣ e.g., $1000 / 6 = 166$, remainder 4
 - ▣ If any errors lead to 996, remainder won't be the same
- CRC: a bit more complicated than that
 - ▣ Generator G : $(r+1)$ -bit pattern known by sender and receiver
 - ▣ Choose r additional bits to append to data D , so that the resulting numbers is divisible by G using modulo-2
 - ▣ CRC code is the r additional bits



Error Detection: CRC

15

Sender wants to compute R such **G**

that:

$$D \cdot 2^r \text{ XOR } R = nG$$

... or equivalently (XOR R both sides):

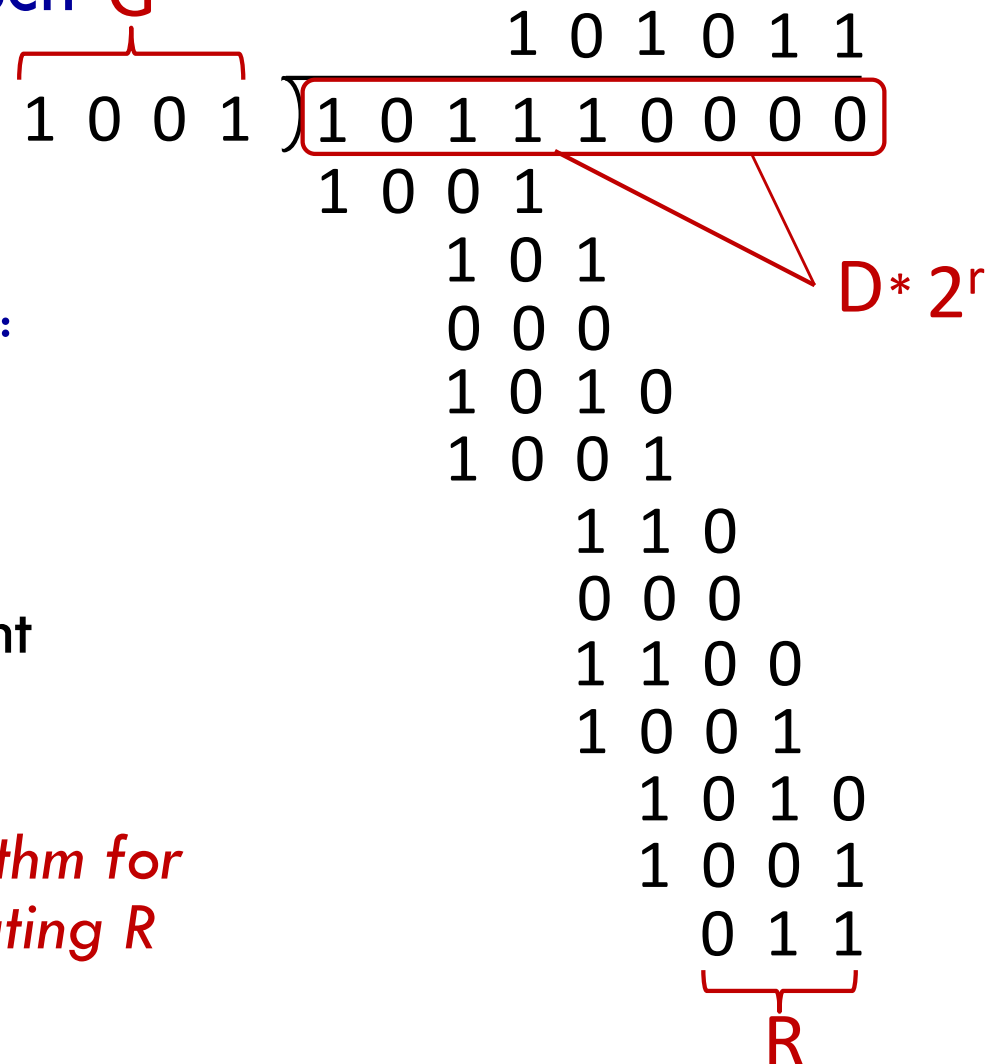
$$D \cdot 2^r = nG \text{ XOR } R$$

... which says:

if we divide $D \cdot 2^r$ by G , we want remainder R to satisfy:

$$R = \text{remainder} \left[\frac{D \cdot 2^r}{G} \right]$$

algorithm for
computing R



Quiz

16

- Consider a network topology with three nodes shown in the right. All nodes are running the distance vector routing algorithm to find the shortest path to other nodes.

How many rounds does it take for the algorithm to converge? Feel free to draw the distance vector tables at each round.

